Ders 5 Genişletilmiş Notu: Matematiksel Analiz İçin Formüller, Özyinelemesiz ve Özyinelemeli Algoritmaların Analizi

Ders 5 Genişletilmiş Notu: Matematiksel Analiz İçin Formüller, Özyinelemesiz ve Özyinelemeli Algoritmaların Analizi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Giriş: İki Temel Yaklaşım
- 2. Matematiksel Analiz İçin Formüller
- 3. İteratif ve Özyinelemeli Çözümlerin Farkları
- 4. Özyinelemesiz Algoritmaların Analizi: MaxElement
- 5. Özyinelemesiz Algoritmaların Analizi: UniqueElements
- 6. Çalışma Süresi Tahmini
- 7. Özyinelemeli Algoritmaların Analizi: Faktöriyel
- 8. Özyinelemeli Algoritmaların Analizi: İkilik Basamak Sayısı (BinRec)
- Örnek Sorular
- Dersin Toparlaması

1. Giriş: İki Temel Yaklaşım

Bu kayıt, dönemin üçüncü haftasının konu anlatımı kısmıdır. Hafta, iki saat konu ve ardından bir saat uygulama düzeniyle yürütülüyor. Önceki derslerde asimptotik gösterimler (asymptotic notations), en kötü, ortalama ve en iyi durum (worst, average, best case) senaryoları ve algoritma verimliliğini ölçme yöntemleri konuşulmuştu. Bu derste bu yöntemlerden **matematiksel analiz** üzerinden ilerleniyor.

Hoca derse bir soruyla başladı: bilgisayar biliminde algoritmik problemler genelde hangi iki temel yaklaşımla çözülür? Cevap **iteratif** (yinelemeli) ve **rekürsif** (özyinelemeli) çözümlerdir. Daha alt başlıklara inildiğinde böl ve yönet (divide and conquer), dinamik programlama, graf algoritmaları ve kombinatorik problemler gibi özel yaklaşımlar gelecektir. Dönem boyunca bu alt başlıklar üzerine ilerlenecek.

Bu dersin iki sorusu şunlardır:

- İteratif algoritmaların matematiksel analizi nasıl yapılır?
- Rekürsif algoritmaların matematiksel analizi nasıl yapılır?

Bunlara geçmeden önce, analizde sık kullanılacak formüller hatırlatıldı. Kaydın başındaki birkaç dakika ses sorunuyla geçti; asıl anlatım yaklaşık 00:07:50'de başlar.

2. Matematiksel Analiz İçin Formüller

Hoca formüllerin çoğunun bilindiğini, ama kullanılmayınca unutulabildiği için hatırlatmakta fayda olduğunu söyledi ve not alınmasını istedi.

2.1 Logaritma Kuralları

İlk olarak logaritmik açılımlar tahtaya yazıldı. Hoca bunların bir kısmının ilerleyen derslerde kullanılacağını söyledi.

> **Ek bilgi (derste anlatılmadı):** Bu tür listelerde büyük olasılıkla şu temel kurallar yer alır: $\log_a 1 = 0$, $\log_a a = 1$, $\log_a x^y = y \log_a x$, $\log_a xy = \log_a x + \log_a y$, $\log_a \frac{x}{y} = \log_a x - \log_a y$, $a^{\log_b x} = x^{\log_b a}$ ve taban değiştirme kuralı $\log_a x = \frac{\log_b x}{\log_b a}$. Taban değiştirme kuralı sayesinde asimptotik analizde logaritmanın tabanı önemsizdir; tabanlar arasında yalnız sabit bir çarpan fark vardır.

2.2 Kombinatorik

Kombinatorik problemler için üç temel ifade hatırlatıldı. Hoca bunları sınıfa sordu.

- **Permütasyon:** n elemanlı bir kümenin permütasyon sayısı $P(n) = n!$'dir. Bir öğrenci "n faktöriyel" diye doğru cevap verdi.
- **Kombinasyon:** n elemanlı bir kümeden k elemanlı seçimlerin sayısı

$$C(n,k) = \frac{n!}{k!\,(n-k)!}$$

Hoca bunun istatistik dersinden de bilinmesi gerektiğini söyledi. - **Alt küme sayısı:** n elemanlı bir kümenin alt küme sayısı 2^n'dir.

2.3 Toplam Formülleri

Sonraki kısım, ileride çözümlerde sürekli kullanılacak toplam formülleridir (summation formulas).

Formül 1. i'nin l'den başlayıp u'ya kadar gittiği bir toplamda her turda 1 eklenirse $u - l + 1$ tane bir toplanır:

$$\sum_{i=l}^{u} 1 = u - l + 1, \quad l \le u$$

Hoca bu formülün n elemanlı işler yaparken çok sık kullanılacağını söyledi. Döngü sayılarını bulmanın temel aracı budur.

Formül 2. Bu kez her turda bir değil i kadar iş yapılsın, yani $1, 2, 3, \ldots, n$ toplanır:

$$\sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2} \approx \frac{1}{2} n^2$$

Formül 3. Kareler toplamı:

$$\sum_{i=1}^{n} i^2 = 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6} \approx \frac{1}{3} n^3$$

Formül 4. Üs 1 ve 2 için görülen formüller k üssüne genelleştirilir:

$$\sum_{i=1}^{n} i^k = 1^k + 2^k + \cdots + n^k \approx \frac{1}{k+1} n^{k+1}$$

Formül 5. Geometrik toplam. Burada a bir sabittir ve toplam $i = 0$'dan başlar:

$$\sum_{i=0}^{n} a^i = 1 + a + a^2 + \cdots + a^n = \frac{a^{n+1} - 1}{a - 1}, \quad a \neq 1$$

a yerine 2 konursa özel durum elde edilir:

$$\sum_{i=0}^{n} 2^i = 2^{n+1} - 1$$

Formül 6.

$$\sum_{i=1}^{n} i\, 2^i = 1 \cdot 2 + 2 \cdot 2^2 + \cdots + n \cdot 2^n = (n-1)2^{n+1} + 2$$

Formül 7. Harmonik toplam:

$$\sum_{i=1}^{n} \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \approx \ln n + \gamma, \quad \gamma \approx 0{,}5772$$

Hoca ilk beş formülün çok sık, son ikisinin ise daha az kullanılacağını söyledi.

> **Ders sonrası not:** Formüller Python ile kontrol edildi. $n = 1, 5, 10, 100$ için Formül 1, 2, 3, 5 ($a = 2$ ve $a = 3$) ve 6 tam olarak tutar. Örneğin $n = 10$ için Formül 6'nın iki tarafı da 18434 verir. Formül 4 için $k = 3$, $n = 1000$ alınınca $\sum i^3$ ile $\frac{n^4}{4}$ oranı 1,002'dir. Formül 7 için $n = 1000$'de toplam 7,4855, yaklaşık değer 7,4850'dir.
>
> **Ek bilgi (derste anlatılmadı):** Formül 7'deki $\gamma \approx 0{,}5772$ sabiti Euler-Mascheroni sabiti olarak bilinir. Harmonik toplamın $\ln n$ gibi büyüdüğünü, yani $\Theta(\log n)$ olduğunu gösterir.

2.4 Toplam Üzerinde İşlem Kuralları

Algoritmaları çözümlerken birbirinin içine girmiş toplam ifadeleri yazılacak. Bunları ayırarak çözmek için üç temel kural (sum manipulation rules) kullanılır.

1. **Sabit dışarı alınır:**

$$\sum_{i=l}^{u} c\, a_i = c \sum_{i=l}^{u} a_i$$

Hoca bu kuralın çok sık kullanılacağını söyledi. 2. **Toplam ya da fark ayrılır:** İçeride iki işlem varsa ayrı ayrı çözmek daha rahattır:

$$\sum_{i=l}^{u} (a_i \pm b_i) = \sum_{i=l}^{u} a_i \pm \sum_{i=l}^{u} b_i$$

3. **Aralık bölünür:** Toplam belirli bir m noktasından ikiye ayrılır:

$$\sum_{i=l}^{u} a_i = \sum_{i=l}^{m} a_i + \sum_{i=m+1}^{u} a_i$$

> **Ders sonrası not:** Hoca 3. kuralın koşulunu "$l < u$" diye söyledi. Bölme noktası için doğru koşul $l \leq m < u$'dur.

2.5 Alta ve Üste Yuvarlama

Son olarak alta yuvarlama (floor, $\lfloor x \rfloor$) ve üste yuvarlama (ceiling, $\lceil x \rceil$) hatırlatıldı. Hoca alta yuvarlama işaretini gösterip sınıfa sordu. Pozitif bir sayı için öğrenciler 3, negatif karşılığı için -4 cevabını verdi; hoca ikisini de doğru buldu.

Üste yuvarlama bunun tersidir:

$$\lceil 2{,}7 \rceil = 3, \qquad \lceil -2{,}7 \rceil = -2$$

Karşılaştırma için aynı sayıların alta yuvarlanmış hali $\lfloor 2{,}7 \rfloor = 2$ ve $\lfloor -2{,}7 \rfloor = -3$'tür (ara adım eklendi). Negatif sayılarda alta yuvarlamanın sıfırdan uzaklaştırdığına, üste yuvarlamanın sıfıra yaklaştırdığına dikkat edilmelidir.

Ardından floor ve ceiling işlemlerinin kullanım kuralları tahtaya yazıldı. Hoca bunları ezberlemenin beklenmediğini, ama soru çözdükçe alışkanlık kazanılacağını ve büyük kısmını akılda tutmanın faydalı olacağını söyledi.

3. İteratif ve Özyinelemeli Çözümlerin Farkları

Genel matematik kurallarından sonra hoca "özyinelemesiz algoritmaların matematiksel analizi" (mathematical analysis for non-recursive algorithms) başlığını açtı ve önce iteratif ile rekürsif çözümlerin farklarını sordu. Öğrencilerden ve hocadan şu noktalar çıktı:

- **Yığın kullanımı:** Rekürsif algoritmalar yığın (stack) kullanır. Yığının boyutu sınırlıdır. Her çağrı bellekte yeni bir yer açar ve değişken tanımları içerir. Fonksiyon kendini çok fazla çağırırsa bir süre sonra yığın dolar. Hoca bunu önemli farklardan biri olarak saydı.
- **Yapı:** Rekürsif çözüm kendini çağıran (dallanan) fonksiyon kullanır, iteratif çözüm döngü (loop) kullanır.
- **Hız ve yer:** Bir öğrenci iteratif fonksiyonların daha hızlı çalıştığını ve daha az yer kapladığını söyledi. Hoca bunun tamamen genelleştirilemeyeceğini belirtti. Rekürsif çözümde her turda bir fonksiyon çağrısı (function call) yapıldığı için hem bellek hem de çağrı süresi açısından ek yük (overhead) vardır. Bu yüzden birçok problemde rekürsif çözüm biraz daha fazla yer kaplar ve biraz daha yavaştır. Ama bu "iteratif çözüm her zaman daha hızlıdır" demek değildir.
- **Uygulanabilirlik:** Bir problemde iki çözüm de uygulanabilir. Bazen tek çözüm vardır, bazen biri yerine diğerini kurmak tercih edilir.
- **Okunabilirlik ve kod uzunluğu:** Rekürsif algoritmaların ifade edilmesi ve anlaşılması iteratif çözümlere göre daha zordur. Ama çoğu zaman çok daha kısa kodla problem çözülebilir.

  Ders sonrası not: Hoca bir öğrencinin cevabını özetlerken "non-rekürsif çözümlerde stack de kullanılabilir ama heap bölgesinde kullanıyoruz" dedi. Bu doğru değil. İteratif çözümde de yerel değişkenler yığındadır. Fark, rekürsif çözümde her çağrı için yığında yeni bir çerçeve (stack frame) açılmasıdır. Heap yalnız dinamik bellek ayrıldığında kullanılır ve bu iki yaklaşımı ayıran bir özellik değildir.

4. Özyinelemesiz Algoritmaların Analizi: MaxElement

4.1 Sözde Kod

İlk örnek, bir dizideki en büyük elemanı bulma algoritmasıdır. Sözde kod (pseudocode) bir öğrenciyle birlikte adım adım yazıldı: önce bir değişkene dizinin ilk elemanı atanır, sonra dizi bir `for` döngüsüyle gezilir, her elemanın o ana kadarki en büyükten büyük olup olmadığına `if` ile bakılır, büyükse değişken güncellenir, en sonda değişken döndürülür.

```
MaxElement(A[0..n-1])
  maxval <- A[0]
  for i <- 1 to n-1 do
    if A[i] > maxval
      maxval <- A[i]
  return maxval
```

Öğrenci `end for` ve `end if` yazmayı önerdi. Hoca buna itirazı olmadığını, ama burada blokların başlangıç ve bitişini girinti (indentation) ile gösterdiğini söyledi.

4.2 Akış Diyagramı

Hoca aynı algoritmayı farkı vurgulamak için bir kez de akış diyagramıyla (flowchart) çizdi: ilk elemanın atanması, i'yi 1'den başlatıp birer artırarak $n-1$'e kadar götüren döngü, döngü içinde $A[i] > max$ karar kutusu, doğru dalda atama, yanlış dalda hiçbir şey yapılmaması ve en sonda max'ın döndürülmesi. Sözde kod her zaman daha hızlı ifade imkânı verdiği için sonraki problemlerde iki gösterimden yalnız biri, genelde sözde kod, kullanılacak.

4.3 Analizin Adımları

Hoca matematiksel analiz için hangi bilgilere ihtiyaç olduğunu sordu. Bir öğrenci "döngülere bakarız" dedi. Hoca bunun yanlış olmadığını, ama ilk bakılacak yerin başka olduğunu söyledi.

1. **Girdi boyutu:** İlk bakılacak şey eleman sayısıdır, yani n (number of elements).
2. **Temel işlem (basic operation):** En çok tekrar eden iş aranır. Temel işlem olabilecek işlemler:
   - atamalar (assignments),
   - aritmetik işlemler (arithmetic operations),
   - karşılaştırmalar (comparisons).
3. **Temel işlemin yeri:** Temel işlem döngülerde, daha belirli olarak **en içteki döngüde** (innermost loop) aranır.

Neden en içteki döngü? Önceki derslerde her işlem tek tek sayılmıştı: atamalar, döngü dışındakiler, iç döngü dışındakiler, dış döngü içindekiler. Bunların hepsi toplanabilir. Ama analizi genelleştirmek ve en çok yapılan işlemi bulmak istendiği için doğal olarak en içteki döngüye bakılır.

MaxElement'te tek döngü vardır. Karmaşıklık hızlıca $O(n)$ diye söylenebilir. Hoca sonra üç durumun (en kötü, ortalama, en iyi) aynı olup olmadığını sordu. Bir öğrenci önce "değildir" dedi, sonra düzeltti: dizinin sonuna kadar her seferinde gitmek gerektiği için üç durum aynıdır.

4.4 Temel İşlemin Seçimi

Hoca temel işlemi sordu. Bir öğrenci "çıkarma" dedi; büyüklük kontrolünün arka planda bir çıkarma olduğunu kastediyordu. Hoca bunu çıkarma üzerinden anlatmamak gerektiğini, burada `if` içindeki "büyüktür" kontrolünün, yani bir **karşılaştırmanın** olduğunu söyledi.

İki aday vardır:

- `A[i] > maxval` karşılaştırması,
- `maxval <- A[i]` ataması (bir öğrenci bunu "replacement" olarak adlandırdı).

Soru: Hangisi temel işlem olarak seçilmeli, neden? **Cevap:** Karşılaştırma. Atamaya geçmeden önce her durumda karşılaştırma yapılması gerekir. Karşılaştırma her turda yapılır, atama ise yalnız karşılaştırma doğru çıktığında yapılır.

4.5 Toplam ve Sonuç

Karşılaştırma sayısı $C(n)$ ile gösterilir (comparison'ın C'si). i 1'den başlar, $n-1$'e kadar gider ve her turda temel işlem bir kez yapılır:

$$C(n) = \sum_{i=1}^{n-1} 1 = (n-1) - 1 + 1 = n - 1 \in \Theta(n)$$

Burada Formül 1 kullanıldı (ara adım eklendi). C'nin altına durum yazılmadı. Gerektiğinde C_{worst}, C_{avg} ya da C_{best} yazılır. Burada yazılmamasının nedeni, üç durum için geçerli tek bir genel ifadeyle analiz yapılabilmesidir.

4.6 İç İçe ve Arka Arkaya Döngüler

Soru: En içteki döngü n'ye kadar değil de örneğin her seferinde 10 kez dönüyorsa, onu döngü olarak saymalı mıyız? Bunun gibi birkaç şey iç içe olduğunda iki `for` olarak mı, tek `for` olarak mı almalıyız? **Cevap:** İç içe `for` varsa her toplam işareti ($\sum$) bir `for`'a karşılık gelir. MaxElement'te tek toplam işareti vardır. İç içe `for` sayısı arttıkça toplam işaretleri arka arkaya dizilir ve temel işlem en içteki `for` içinden seçilir. Ama iki `for` arka arkaya olsaydı, örneğin biri minimum, diğeri maksimum bulsaydı, birinin işlemi temel işlem seçilirdi ve karmaşıklık yine $O(n)$ olurdu. Önemli olan döngülerin ya da yapılan işlemlerin iç içe olmasıdır.

> **Hocanın vurgusu:** Temel işlemi belirlemek ve eleman sayısını tespit etmek bizim için çok çok önemli. Durumlar farklıysa analiz her durum için ayrı yapılır; MaxElement'te üç durum aynı karmaşıklığa sahip olduğu için genel $C(n)$ yazıldı.

5. Özyinelemesiz Algoritmaların Analizi: UniqueElements

5.1 Problem ve Durumlar

İkinci algoritma, bir dizideki elemanların hepsinin farklı (unique) olup olmadığını bulur. Bir öğrenci problemi "dizideki her elemandan yalnız bir tane bulunsun istiyoruz" diye özetledi.

Hoca algoritmayı yazmadan önce üç durumun aynı olup olmadığını sordu. **Cevap:** Değildir. En iyi durumda ilk bakılan iki eleman aynıdır ve geri kalanına bakmaya gerek kalmaz. Bu yüzden en iyi durum farklıdır ve ortalama durumun da diğerleriyle aynı olma olasılığı ortadan kalkar.

5.2 Sözde Kod

Bir öğrenci, bir bayrak değişkeni tutup ilk elemandan başlayarak sonraki elemanlarla karşılaştırmayı, eşitlik bulunca değişkeni değiştirip döngüden çıkmayı önerdi. Hoca bunun yapılabileceğini söyledi, ama kitaptaki karşılığını yazdı:

```
UniqueElements(A[0..n-1])
  for i <- 0 to n-2 do
    for j <- i+1 to n-1 do
      if A[i] = A[j] return false
  return true
```

Hoca burada iki iç içe `for` kullanıldığına dikkat çekti. Adım sayısı belli olmayan işler için şimdiye kadar `while` öneriliyordu. **Soru:** Burada `for` nasıl kullanılabildi? **Cevap:** Fonksiyondan `return` ile çıkıldığı için döngüler arada kesilebiliyor. Normalde döngüden `break` ile çıkmamak önerilir. Ama fonksiyon yazarken dönüş noktası belli olduğu için bu tür yazım yanlış değildir. Fonksiyon olarak yazılmıyorsa `while` ile yazmak doğru olur.

Soru: Dizi 0'dan $n-1$'e gidiyorsa son elemanı görebilecek miyiz? **Cevap:** C gibi dillerde diziler sıfırıncı gözden başlar ve $n-1$'e kadar gider. Birden dörde gitmekle sıfırdan üçe gitmek aynı şeydir. $n-1$ numaralı göz n. elemana, sıfırıncı göz birinci elemana karşılık gelir.

İç döngünün sınırları sınıfa soruldu; cevap "$i+1$'den $n-1$'e kadar" oldu. Hoca bunu 1, 2, 3, 4, 5 dizisi üzerinde açıkladı:

- Birinci turda elde 1 vardır. 1, sırayla 2, 3, 4 ve 5 ile karşılaştırılır.
- İkinci turda elde 2 vardır. 2, sırayla 3, 4 ve 5 ile karşılaştırılır.
- Her turda eldeki eleman soldan sağa değişir. İç döngü, eldeki elemandan bir sonraki elemandan başlar.

Bir eşitlik bulunduğunda fonksiyon `return false` ile döner. Döngülerden hiç `return false` ile çıkılmazsa sonunda `return true` döner.

5.3 Toplamın Kurulması

Durumlar aynı olmadığı için hangi durumun hesaplanacağı soruldu.

> **Hocanın vurgusu:** Aksi belirtilmediği sürece her zaman en kötü durum (worst case) hesaplanır.

En kötü durum, dizide eşit eleman olmadığı ya da yalnız son iki elemanın eşit olduğu durumdur; o zaman bütün karşılaştırmalar yapılır. İki iç içe döngü olduğu için iki toplam işareti kullanılır. İç toplam $j = i + 1$'den $n - 1$'e, dış toplam $i = 0$'dan $n - 2$'ye gider. Temel işlem yine karşılaştırmadır ($A[i] = A[j]$ kontrolü). Bu yüzden C harfi kullanılır. Hoca bunun genel bir gelenek olduğunu, zorunlu olmadığını söyledi. Her turda bu karşılaştırma bir kez yapılır:

$$C_{worst}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

> **Hocanın vurgusu:** Bu algoritmanın karmaşıklığını hızlıca söyleyebilirsiniz. Ama bizden istenen, bunu adım adım matematiksel olarak göstermektir. Bu dersin amacı da matematiksel analizin adım adım nasıl yapıldığını göstermektir.

5.4 Adım Adım Çözüm

Adım 1: İç toplam. Her zaman önce iç toplam açılır. Bir öğrenci önce "$n - i - 2$" dedi. Hoca yazılan formüllerin kullanılmasını istedi ve sınıfla birlikte Formül 1 uygulandı: üst sınır $n - 1$, eksi alt sınır $(i + 1)$, artı 1:

$$\sum_{j=i+1}^{n-1} 1 = (n-1) - (i+1) + 1 = n - 1 - i$$

Adım 2: Dış toplamı ayırma. Kural 2 ile toplam ikiye ayrılır:

$$C_{worst}(n) = \sum_{i=0}^{n-2} (n - 1 - i) = \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i$$

Adım 3: Sabiti dışarı alma. Kural 1 ile $n - 1$ dışarı alınır. İçeride 1 kalır. i, 0'dan $n - 2$'ye kadar $n - 1$ değer alır:

$$\sum_{i=0}^{n-2} (n-1) = (n-1) \sum_{i=0}^{n-2} 1 = (n-1)(n-1)$$

Adım 4: İkinci toplam. Bir öğrenci $\sum_{i=0}^{n-2} i = \frac{(n-2)(n-1)}{2}$ dedi. Gerekçesi: üst sınır n olsaydı $\frac{n(n+1)}{2}$ yazılacaktı; bu, üst sınır $n - 2$'ye göre uyarlanırsa $\frac{(n-2)(n-1)}{2}$ olur. $i = 0$ terimi toplama bir şey eklemez.

Adım 5: Sonuç.

$$C_{worst}(n) = (n-1)^2 - \frac{(n-2)(n-1)}{2} = \frac{(n-1)(2(n-1) - (n-2))}{2} = \frac{(n-1)n}{2} \approx \frac{1}{2}n^2 \in \Theta(n^2)$$

Ortadaki çarpanlara ayırma adımı eklendi (ara adım eklendi).

> **Ders sonrası not:** Öğrencinin ilk cevabı olan "$n - i - 2$" yanlıştır; derste de düzeltildi. Doğrusu $n - 1 - i$'dir. Python ile kontrol: $n = 2, \ldots, 11$ için iç içe döngünün gerçek karşılaştırma sayısı her zaman $\frac{n(n-1)}{2}$'ye ve $(n-1)^2 - \frac{(n-2)(n-1)}{2}$'ye eşittir. Örneğin $n = 5$ için 10 karşılaştırma yapılır; "$n - i - 2$" ile kurulan toplam ise 6 verirdi.

5.5 Matris Çarpımı Sorusu

Hoca kısa bir soru daha sordu: matris çarpımında kaç toplam işareti kullanmak gerekir? Cevap üçtür, çünkü matris çarpımı üç iç içe döngüyle yapılır.

6. Çalışma Süresi Tahmini

Hoca önemli bir nokta daha ekledi ve çalışma süresini şöyle yazdı:

$$T(n) \approx c_{op}\, C(n)$$

UniqueElements için bu $T(n) \approx c\, n^2$ gibidir. **Soru:** Buradaki küçük c neye karşılık gelir? **Cevap (öğrenci):** Her döngünün ya da atamanın bilgisayarda farklı bir maliyeti (cost) vardır. Bunlar c_1, c_2 gibi ayrı ifade edilebilirdi, ama sonunda toplam bir sabit olacağı için tek bir c yazılır. Hoca genel olarak bunu doğru buldu ve şöyle netleştirdi: küçük c, temel işlemin maliyetidir.

Bu durumda $T(n)$, temel işlemin bir kez yapılma süresinin, analizle bulunan temel işlem sayısıyla çarpılmasıdır. Yani algoritmanın bir bilgisayar üzerindeki çalışma süresinin (running time) yaklaşık değeridir. Bu çok genel bir yazımdır. Temel işlemin yanında diğer işlemler de katılarak daha uzun bir ifade yazılabilir, ama analiz genelde temel işlem üzerinden yapılır.

Gösterim temel işleme göre değişir:

- Karşılaştırma için c_c ve $C(n)$: c_c, ilgili cihazdaki karşılaştırma komutunun işlem süresidir (the time of one comparison).
- Çarpma (multiplication) için c_m ve $M(n)$.
- Toplama (addition) için c_a ve $A(n)$.

7. Özyinelemeli Algoritmaların Analizi: Faktöriyel

7.1 Problem ve İlk Çözüm

İteratif problemlerden sonra aynı işlem rekürsif algoritmalar için yapıldı. Bilinen en güzel örnek faktöriyel hesabıdır:

$$n! = 1 \cdot 2 \cdots (n-1) \cdot n = (n-1)! \cdot n, \quad n \geq 1$$

Bu problem iteratif olarak da rekürsif olarak da çözülebilir; ikisi de doğrudur. Rekürsif çözüm fonksiyonun kendini çağırmasıdır ve çok daha az kod yazmayı sağlar.

Bir öğrenci rekürsif kodu şöyle önerdi: n 1 ise 1 döndür, değilse n ile `Factorial(n-1)`'in çarpımını döndür. Buradaki `if n = 1` satırı durma noktasıdır.

7.2 $n = 0$ Durumu

Soru: Bu çözüm doğru mu? **Cevap:** Bir öğrenci $n = 0$ durumuna da bakmak gerektiğini söyledi. Hoca bunu doğruladı: bu çözüm $n = 0$ durumunu dikkate almıyor. $0! = 1$ olduğu bilinir, ama bu kodda `F(0)` çağrısı durma koşuluna hiç ulaşmaz ve sonsuz özyinelemeye girer.

Çözümün nasıl düzeltileceği soruldu. Bir öğrenci `n = 1` yerine `n < 1` yazmayı, negatif sayılar için de başa ayrı bir `if` koymayı önerdi; hoca ek bir `if`'e gerek olmadığını söyledi. Başka bir öğrenci `n = 1` yerine `n = 0` yazmayı önerdi. Hoca bunu birlikte denedi: `F(1)` çağrıldığında $1 \cdot F(0)$ hesaplanır, `F(0)` çağrıldığında `if n = 0` doğru olduğu için 1 döner. Böylece $0!$ de doğru hesaplanır ve çözüm doğru çalışır.

```
F(n)
  if n = 0 return 1
  else return F(n-1) * n
```

Genelleştirmek için $F(n) = F(n-1) \cdot n$, $n > 0$ yazıldı.

> **Ders sonrası not:** Düzeltilmiş kod Python ile çalıştırıldı: $F(0), \ldots, F(5)$ için 1, 1, 2, 6, 24, 120 elde edildi.

7.3 Temel İşlem ve Rekürans Bağıntısı

Temel işlem sorulduğunda öğrenciler "çarpma" dedi. Bu yüzden C yerine M (multiplication) kullanılır. Hoca $M(n) = M(n-1)$'den sonra ne yazılacağını sordu. Bir öğrenci yine çarpı n yazılıp yazılmayacağını sordu. Hoca hayır dedi: $F(n) = F(n-1) \cdot n$ işin nasıl yapıldığını gösterir. Matematiksel ifadede ise temel işlemin sayısı yazılır. Her seferinde bir çarpma yapıldığı için $+1$ eklenir:

$$M(n) = M(n-1) + 1, \quad n > 0$$

- $M(n-1)$: $F(n-1)$'i hesaplamak için yapılan çarpmalar.
- $+1$: $F(n-1)$'i n ile çarpmak için yapılan tek çarpma.

Bu ifadeyi çözmek için bir **rekürans bağıntısı** (recurrence relation) yazılır ve kurulur. Rekürans bağıntısını yazarken bir **başlangıç koşuluna** (initial condition) ihtiyaç vardır.

7.4 Başlangıç Koşulu

Başlangıç koşulu, algoritmanın durma noktasını belirleyen ifadedir. Burada bu ifade `if n = 0 return 1` satırıdır. Bu satır matematiksel olarak bir fonksiyon değeriyle yazılmalıdır.

Soru: $M(0)$ kaçtır? Bir öğrenci 1 dedi; hoca bunun doğru olmadığını söyledi. **Cevap:** Durma noktasında yalnız `return` yapılır, hiçbir çarpma yapılmaz. Bu yüzden:

$$M(0) = 0 \quad \text{(no multiplications when } n = 0)$$

Toparlanırsa:

$$M(n) = M(n-1) + 1 \ (n > 0), \qquad M(0) = 0$$

7.5 Geriye Doğru Yerine Koyma

Rekürans bağıntısını çözmenin yöntemlerinden biri **geriye doğru yerine koyma** yöntemidir (method of backward substitution). Hoca bunun çok kolay bir yaklaşım olduğunu, geriye dönük analiz yapıldığını söyledi.

$M(n-1)$ yerine $M(n-2)+1$ konur, sonra $M(n-2)$ yerine $M(n-3)+1$ konur:

$$\begin{aligned} M(n) &= M(n-1) + 1 \\ &= [M(n-2) + 1] + 1 = M(n-2) + 2 \\ &= [M(n-3) + 1] + 2 = M(n-3) + 3 \\ &\ \ \vdots \\ &= M(n-i) + i \end{aligned}$$

Bir öğrenci genel ifadeyi $M(n-k)+k$ diye verdi; hoca k yerine i kullanacağını, ama k'nin de yanlış olmadığını söyledi.

Sonraki adımı bir öğrenci açıkladı: $M(0)$ bilindiği için $n - i$'yi sıfıra eşitlemek gerekir, yani $i = n$ alınır:

$$M(n) = M(n-n) + n = M(0) + n = 0 + n = n$$

Buna göre algoritmanın karmaşıklığı n'dir, yani $M(n) \in \Theta(n)$.

> **Ders sonrası not:** $M(n) = M(n-1) + 1$, $M(0) = 0$ rekürsif olarak Python'da hesaplandı; $n = 0, \ldots, 19$ için her zaman $M(n) = n$ çıktı.

7.6 Soru-Cevaplar

Soru: Durma koşulundaki `n = 0` karşılaştırması temel işlem sayılmıyor, onun yerine çarpma sayılıyor. Neden? **Cevap:** Her seferinde yapılan iş, fonksiyonun kendini çağırması ve o çağrıda yapılan çarpmadır. Durma koşulundaki karşılaştırma birçok yerde durma noktası, yani başlangıç koşulu olarak kullanılacak. Rekürsif fonksiyonlarda genelde fonksiyonun çağrıldığı satırdaki işlem temel işlem olarak alınır.

Soru: $+1$ temel işlem sayısı mı? Toplama ya da iki işlem olsaydı $+2$ mi yazardık? **Cevap:** Genelde bunların içinden bir tanesi seçilir. Temel işlem için ya sabit olarak 1, ya da parametre cinsinden i, n veya k kullanılır; 2, 3, 4 gibi sabitler yazılmaz.

> **Ders sonrası not:** Hocanın cevabı uygulamadaki geleneği anlatıyor. Matematiksel olarak her çağrıda c sabit sayıda işlem yapılıyorsa $M(n) = M(n-1) + c$ yazılabilir. Sonuç $M(n) = cn$ olur ve büyüme derecesi yine $\Theta(n)$'dir.

Soru: Fonksiyon çağırma işlemi temel işlem olarak alınabilir miydi? **Cevap:** Genelde alınmaz. Matematiksel analizde temel işlem genelde atama, dört işlem ya da karşılaştırmadır. Fonksiyon çağrısında mutlaka yapılan bir iş vardır ve o iş temel işlem olarak seçilir.

8. Özyinelemeli Algoritmaların Analizi: İkilik Basamak Sayısı (BinRec)

8.1 Problem

Son örnek, verilen bir sayının ikilik (binary) gösteriminde kaç basamak olduğunu rekürsif olarak bulmaktır. Sayının pozitif olduğu kabul edilir.

Hoca önce "ikilik sayı sisteminde verilen bir sayı" dedi. Bir öğrenci buna göre sayıyı 1 ile VE'leyip (AND) sağa kaydırarak (shift right) özyinelemeli çağrı yapan bir çözüm önerdi. Hoca sonra yanlış ifade ettiğini düzeltti: onluk sistemde verilen 8, 7, 29 gibi bir sayının ikilik karşılığının kaç basamaktan oluştuğu bulunacak. Öğrenci buna göre de "bir sağa kaydırıp bir artırırız" dedi; ikilik gösterimde sağa kaydırma 2'ye bölmeye denktir.

8.2 Sözde Kodun Kurulması

Başka bir öğrenci 2'ye bölerek ilerlemeyi önerdi. Önerisi önce "n sıfırsa 1 döndür, değilse $n/2$ ile kendini çağır" şeklindeydi. Hoca kendini çağırırken neyin gönderileceğini sordu; cevap bölümün gönderilmesiydi. Ardından her çağrıda bir basamak ilerlendiği için sonuca $+1$ eklenmesi önerildi.

Durma koşulu üzerine farklı öneriler geldi:

- Bir öğrenci `if n = 0` satırında `return 1` yerine `return 0` olması gerektiğini düşündü.
- Başka bir öğrenci `n = 0` için 0, `n = 1` için 1 döndürmeyi önerdi.

Hoca, sayı ikiye bölüne bölüne ilerlerken son noktanın genelde 1 olacağını ve `if n = 1 return 1` yazmanın sonuca doğrudan ulaştırdığını söyledi. `return 0` ile son adımda bir değer döndürülmemiş olur.

```
BinRec(n)            // n pozitif tam sayı
  if n = 1 return 1
  else return BinRec(floor(n/2)) + 1
```

> **Ders sonrası not:** Öğrencinin ilk önerdiği `if n = 0 return 1` durma koşulu yanlıştır: `BinRec(1)`, `BinRec(0) + 1 = 2` döndürür ve her sayı için bir fazla sayar. Hocanın son hali doğrudur. Python ile 1, 7, 8, 29 için doğru basamak sayıları (1, 3, 4, 5) elde edildi; $n = 1, \ldots, 69$ için sonuç her zaman ikilik gösterimin uzunluğuna eşittir.

8.3 Temel İşlem

Soru: Temel işlem nedir? Bölme olabilir mi? **Cevap:** Bölme değildir; bölme yapılıp sonucu parametre olarak gönderilir. Her turda asıl yapılan iş toplamadır (addition). Bu yüzden $A(n)$ yazılır.

Soru: Karşılaştırma olmayacak mı? **Cevap:** Hayır. Karşılaştırma başlangıç koşuluna karşılık gelir. Asıl ilgilenilen, basamak sayısını bulmak için yapılan toplama işlemidir.

8.4 Rekürans Bağıntısı ve Başlangıç Koşulu

Bir öğrenci faktöriyele bakarak $A(n-1)+1$ yazmayı önerdi. Hoca burada $n-1$ değil $n/2$ gönderildiğini hatırlattı:

$$A(n) = A(\lfloor n/2 \rfloor) + 1, \quad n > 1$$

Başlangıç koşulu `if n = 1 return 1` satırından gelir. Bir öğrenci önce $A(1) = 1$ dedi, sonra düzeltti: durma noktasında toplama yapılmadığı için

$$A(1) = 0$$

8.5 Düzgünlük Kabulü ve Çözüm

Çözümü kolaylaştırmak için düzgünlük (smoothness) yaklaşımı kullanılır: n'nin 2'nin kuvveti olduğu kabul edilir, $n = 2^k$. Hoca algoritma çalışırken 2'nin kuvvetleriyle iş yapıldığında sonucun çok daha rahat ifade edildiğini, bunun ileride de görüleceğini söyledi. n 2'nin kuvveti olmadığında sonucun yanlış çıkacağı anlamına gelmez; bu kabul yalnız anlamayı kolaylaştırmak içindir.

$n = 2^k$ için $\lfloor n/2 \rfloor = 2^{k-1}$ olur:

$$A(2^k) = A(2^{k-1}) + 1, \quad k > 0, \qquad A(2^0) = 0$$

Geriye doğru yerine koyma:

$$\begin{aligned} A(2^k) &= A(2^{k-1}) + 1 \\ &= [A(2^{k-2}) + 1] + 1 = A(2^{k-2}) + 2 \\ &= [A(2^{k-3}) + 1] + 2 = A(2^{k-3}) + 3 \\ &\;\;\vdots \\ &= A(2^{k-i}) + i \end{aligned}$$

Çözümü tamamlamak için $i = k$ alınır:

$$A(2^k) = A(2^{k-k}) + k = A(2^0) + k = A(1) + k = 0 + k = k$$

$n = 2^k$ olduğu hatırlanırsa $k = \log_2 n$'dir. Yani

$$A(n) = \log_2 n \in \Theta(\log n)$$

Onluk bir sayının ikilik basamaklarını sayan rekürsif çözümün karmaşıklığı $\log n$'dir.

> **Ders sonrası not:** Python ile kontrol: $k = 0, \ldots, 5$ için $A(2^k) = k$. Genel n için ($n = 1, \ldots, 69$) toplama sayısı $A(n) = \lfloor \log_2 n \rfloor$'dir. Algoritmanın döndürdüğü basamak sayısı ise $\lfloor \log_2 n \rfloor + 1$'dir. Yani $A(n)$ toplama sayısıdır, basamak sayısının kendisi değildir.

8.6 Soru-Cevaplar

Soru: Bu haliyle $n = 0$ verilirse sonsuz döngüye girmez mi? **Cevap:** Hoca "bakacağız" dedi, ama soru derste ayrıca cevaplanmadı.

> **Ders sonrası not:** Evet, `BinRec(0)` hiçbir zaman `n = 1` koşuluna ulaşmaz ($\lfloor 0/2 \rfloor = 0$) ve sonsuz özyinelemeye girer. Algoritma yalnız pozitif tam sayılar için tanımlıdır; problem tanımında da girdi pozitif kabul edilmişti.

Soru: Faktöriyelde $M(0) = 0$, burada $A(1) = 0$ dedik. Başlangıç koşulu hep sıfır mı olur? **Cevap:** Her zaman sıfır olmayacak. Burada yapılan iş toplamadır ve fonksiyon her kendini çağırdığında bu toplama yapılır. Fonksiyon sonlanırken ise yalnız 1 değeri döndürülür. Durma noktasında da bir toplama olsaydı, örneğin orada da `BinRec(n/2) + 1` gibi bir işlem yapılsaydı, $A(1) = 1$ olurdu. Önemli olan, sayılan işlemin durma noktasında yapılıp yapılmadığıdır.

Hoca bu bölümün beklenenden uzun sürdüğünü söyledi ve ara sonrası uygulama saatinde bugünkü konuyla ilgili örnekler çözüleceğini belirtti.

Örnek Sorular

Bu kayıtta kare ve video yok. Sorular, derste sözlü sorulanlardan alındı.

Soru 1 (derste sözlü, 00:25:26). İteratif çözümlerle rekürsif çözümler arasındaki farklar nelerdir?

> **Çözüm:** Rekürsif çözüm kendini çağıran fonksiyon, iteratif çözüm döngü kullanır. Rekürsif çözümde her çağrı yığında yer açar; çok derin özyineleme yığını doldurabilir. Her çağrının bellek ve zaman ek yükü olduğu için rekürsif çözüm çoğu zaman biraz daha yavaş ve daha çok yer kaplar, ama bu her zaman geçerli değildir. Rekürsif çözüm daha kısa koddur, ama anlaşılması daha zordur.

Soru 2 (derste sözlü, 00:39:49 ve 00:40:59). MaxElement algoritmasında temel işlem nedir? `A[i] > maxval` karşılaştırması ile `maxval <- A[i]` atamasından hangisi seçilmeli, neden?

> **Çözüm:** Karşılaştırma seçilir. Karşılaştırma her turda yapılır, atama yalnız karşılaştırma doğruysa yapılır. $C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n)$; üç durum aynıdır.

Soru 3 (derste sözlü, 00:58:15). UniqueElements probleminde en kötü, ortalama ve en iyi durum aynı mıdır?

> **Çözüm:** Değildir. En iyi durumda ilk bakılan iki eleman eşittir ve algoritma tek karşılaştırmayla biter. Bu yüzden en kötü durum ayrıca hesaplanır: $C_{worst}(n) = \frac{(n-1)n}{2} \in \Theta(n^2)$.

Soru 4 (derste sözlü, 01:08:36). $\sum_{j=i+1}^{n-1} 1$ iç toplamının açık hali nedir?

> **Çözüm:** Formül 1 ile $(n-1) - (i+1) + 1 = n - 1 - i$. Derste verilen ilk cevap olan $n - i - 2$ yanlıştır.

Soru 5 (derste sözlü, 01:14:45). Matris çarpımında kaç toplam işareti kullanmak gerekir?

> **Çözüm:** Üç. Derste yalnız sayı söylendi.
>
> **Çözüm (derste verilmedi):** İki $n \times n$ matrisin çarpımında üç iç içe döngü vardır ve en içte bir çarpma yapılır: $M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1 = n^3 \in \Theta(n^3)$. Python ile $n = 1, \ldots, 6$ için üç iç içe döngünün sayısı n^3 çıktı.

Soru 6 (derste sözlü, 01:22:09). `if n = 1 return 1; else return F(n-1) * n` faktöriyel çözümü doğru mudur?

> **Çözüm:** Değildir. `F(0)` çağrısı durma koşuluna hiç ulaşmaz ve sonsuz özyinelemeye girer. Durma koşulu `if n = 0 return 1` olmalıdır; bu hem $0! = 1$'i verir hem de diğer değerleri doğru hesaplar.

Soru 7 (derste sözlü, 01:30:08). Faktöriyelin çarpma sayısı için başlangıç koşulu $M(0)$ kaçtır?

Çözüm: $M(0) = 0$. Durma noktasında yalnız `return 1` yapılır, çarpma yapılmaz. Rekürans $M(n) = M(n-1) + 1$ ile birlikte geriye doğru yerine koyma $M(n) = M(n-i) + i$ verir; $i = n$ alınınca $M(n) = n$ olur.

Soru 8 (derste sözlü, 01:45:31 ve 01:47:33). BinRec algoritmasında temel işlem ve başlangıç koşulu nedir?

Çözüm: Temel işlem özyinelemeli çağrıdan sonraki toplamadır; bölme ve durma koşulundaki karşılaştırma sayılmaz. $A(n) = A(\lfloor n/2 \rfloor) + 1$ $(n > 1)$, $A(1) = 0$. $n = 2^k$ kabulüyle $A(n) = \log_2 n \in \Theta(\log n)$.

Dersin Toparlaması

Bu derste önce analizde kullanılacak araçlar hatırlatıldı: logaritma kuralları, permütasyon, kombinasyon ve alt küme sayısı, yedi toplam formülü, toplam üzerinde üç işlem kuralı ve alta/üste yuvarlama. Ardından özyinelemesiz algoritmaların analizi için bir yol izlendi: girdi boyutu belirlenir, en içteki döngüde temel işlem bulunur, durumların aynı olup olmadığına bakılır, temel işlemin sayısı iç içe döngü başına bir toplam işaretiyle yazılır ve toplam formüllerle kapalı biçime getirilir. MaxElement'te üç durum aynı olduğu için genel $C(n) = n - 1 \in \Theta(n)$ yazıldı. UniqueElements'te durumlar farklı olduğu için en kötü durum adım adım çözüldü ve $\frac{(n-1)n}{2} \in \Theta(n^2)$ bulundu. Çalışma süresi $T(n) \approx c_{op} C(n)$ olarak tahmin edildi.

Özyinelemeli algoritmalarda temel işlem, fonksiyonun kendini çağırdığı satırdaki işlemdir. Bu işlemin sayısı bir rekürans bağıntısıyla yazılır. Durma koşulu başlangıç koşulunu verir ve durma noktasında temel işlem yapılmıyorsa başlangıç değeri sıfırdır. Bağıntı geriye doğru yerine koyma ile çözülür. Faktöriyel için $M(n) = n$, ikilik basamak sayısı için $n = 2^k$ kabulüyle $A(n) = \log_2 n$ bulundu. Aynı günün uygulama saatinde bu konuyla ilgili örnekler çözülecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.